

TP — Réveil musical

Contexte et besoins métier

Votre entreprise lance **Réveil musical**, un service qui réveille chaque utilisateur avec un morceau choisi selon le jour de la semaine et la météo du jour, puis le prévient sur le canal de son choix.

Le produit est encore jeune et la feuille de route produit n'est pas figée :

- **Côté musique**, le fournisseur de départ n'est pas garanti dans la durée : le board veut pouvoir tester plusieurs sources de morceaux et en changer rapidement si l'une s'avère trop chère, trop limitée, ou en panne.
- **Côté notification**, les retours utilisateurs early-access indiquent que tout le monde ne veut pas être réveillé de la même façon : certains veulent un email, d'autres un SMS, d'autres une notification push. De nouveaux canaux (WhatsApp, appel vocal) sont déjà évoqués pour les mois à venir. Pour ce TP, l'envoi réel n'est pas requis : vous simulerez chaque canal par un mock que vous écrivez vous-même.
- **Côté légal**, l'entreprise a été échaudée par un précédent produit où une dépendance mal vérifiée a bloqué une levée de fonds. La direction exige désormais qu'**aucun composant externe ne soit intégré sans vérification préalable de sa licence et de sa fraîcheur**.
- **Côté fiabilité**, le produit envoie un réveil à une heure précise : une panne du fournisseur musical ou du canal de notification ne doit jamais empêcher l'envoi (un mode dégradé est acceptable, un silence ne l'est pas).

Ce qu'on vous demande, en tant qu'équipe technique : livrer une première version qui tient ces quatre exigences, sachant qu'aucune d'elles n'est négociable, mais que leur traduction technique vous appartient entièrement.

Point d'entrée du TP

Le TP porte sur l'appel qui déclenche l'envoi, exécuté à la bonne heure (l'ordonnancement n'est pas à coder). Cet appel reçoit l'**ID utilisateur**, le **jour de la semaine** et le **type de météo du jour** (**SOLEIL** / **PLUIE** / **NEIGE** / **NUAGEUX** — tous fournis en entrée, aucun appel météo externe). Un service interne (à mocker) retourne, à partir de l'ID, le morceau choisi par l'utilisateur pour chaque type de météo, un **morceau de secours** pour les cas non couverts, et le canal préféré — un fournisseur comme un autre, accédé via une interface.

Exigence d'architecture explicite

Au-delà du produit qui doit fonctionner, votre code sera évalué sur la façon dont il traduit ces besoins métier :

- **Isolation et faible couplage** : aucune classe métier ne doit connaître un détail technique d'un fournisseur ou d'un canal particulier.
- **IoC / DI** : aucune implémentation concrète ne doit être instanciée (**new**).

Ces deux points traduisent directement le besoin business « pouvoir changer de fournisseur ou de canal rapidement, sans tout réécrire ».

APIs disponibles

Musique — iTunes Search API (gratuite, sans clé)

```
GET https://itunes.apple.com/search?term=<morceau>&media=music&limit=5
```

- Pas de clé requise ; ~20 requêtes/minute : à respecter côté cache. Réponse JSON avec notamment **trackName**, **artistName** et **trackViewUrl** (champ propre à iTunes — ne doit pas fuiter dans le métier).

Musique — MusicBrainz (gratuite, sans clé)

```
GET https://musicbrainz.org/ws/2/recording?query=<morceau>&fmt=json
```

- Exige un en-tête **User-Agent** identifiable (nom d'application + contact), sinon la requête est rejetée. Réponse JSON avec notamment **title** et **artist-credit**.

Fallback local : une petite liste de morceaux codée en dur, à utiliser quand aucun fournisseur n'est disponible — le silence n'est jamais acceptable.

Notifications — mocks à mettre en place (non fournis) : simuler email, SMS et push (pas d'envoi réel, ex. écriture console ou fichier log), chacun avec une interface volontairement différente — à ramener vers une interface commune via vos adaptateurs.

Livrables attendus

- Un **dépôt Git** contenant le code source complet du projet.
- Un **README.md** listant **chaque package/SDK** avec sa **licence**, sa **version installée** et sa **fraîcheur** (dernière version stable), et une justification si un composant pose question (copyleft, version ancienne...).
- **Des tests unitaires** avec un bon niveau de couverture.